



UNIVERSIDAD NACIONAL DE RÍO CUARTO

SISTEMAS OPERATIVOS

(Código 1965)

Mini-taller 2: EDOS con soporte SMP

Análisis del arranque multiprocesador
y sincronización de salida por UART

Departamento de Computación

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Alumno
Tomás Rodeghiero

Fecha de entrega: 30 de abril de 2026

Índice

Objetivo	3
Entorno y estructura	3
Análisis del código original	3
Ejecución con QEMU	4
1. ¿Por qué aparece la salida mezclada en consola?	4
2. Modificación de la función <code>kernel_main()</code> para evitar la salida mezclada	5
Conclusión	6

Objetivo

El objetivo de este Mini-taller fue leer el código entregado, ejecutarlo en QEMU con dos CPUs y resolver el ejercicio del `README`, que pide modificar `kernel_main()` para que la salida por consola deje de aparecer mezclada.

Lo que me interesaba ver era cómo encajan tres cosas en un mismo lugar: el arranque por hart, los stacks separados por CPU y la sincronización al usar la UART, que es un recurso compartido entre los dos núcleos.

Entorno y estructura

Para compilar y ejecutar usé `make`, la toolchain RISC-V y `qemu-system-riscv32` sobre la máquina `virt`, sin BIOS, con `-smp 2`.

La entrega quedó organizada así:

- `02-hello-smp/`: el paso original con el `kernel.c` *sin modificar* y los archivos generados al compilar.
- `entrega/kernel.c`: la versión corregida.
- `README.md`: explicación general de la entrega.
- `informe-minitaller2-smp.pdf`: este informe.

Decidí dejar la carpeta `02-hello-smp` tal como vino con los archivos compilados. Así, comparando los dos `kernel.c`, se ve a simple vista cuál fue el cambio que terminé aplicando.

Análisis del código original

El punto central del código entregado originalmente es que en RISC-V cada núcleo es un *hart* con su `hartid`, y al encender la máquina todos los harts arrancan ejecutando el mismo código.

En `arch.s` cada hart guarda su `mhartid` en `tp` y calcula su propio `sp` dentro del bloque que reserva el linker script:

Stack pointer por hart, en `arch.s`

```
1 csrr tp, mhartid
2 la sp, __stack0
3 li t0, 1024*4
4 addi t1, tp, 1
5 mul t0, t0, t1
6 add sp, sp, t0
```

`kernel.ld` reserva 4 stacks de 4 KB consecutivos a partir de `__stack0`, y `cpuid()` (definida también en `arch.s`) devuelve el contenido de `tp`, así que desde C se puede saber en qué hart se está ejecutando.

El otro archivo importante que me fue importante considerar es `spinlock.c/h`, tomado de xv6. Lo menciono porque es la pieza que después uso para resolver el ejercicio: implementa `acquire()` y `release()` con la operación atómica `__sync_lock_test_and_set` y barreras `__sync_synchronize`.

Una pista que me llamó la atención al leer `kernel.c`: hay declarada una variable global

```
1 spinlock lk = 0;
```

que no se usa en ningún lado. Eso me terminó de orientar sobre por dónde tenía que ir la corrección.

Ejecución con QEMU

El Makefile ya trae todo armado. En mi caso, con el toolchain instalado por Homebrew, lo corrí así:

```
1 cd 02-hello-smp
2 make PREFIX=riscv64-unknown-elf-
3 make qemu PREFIX=riscv64-unknown-elf-
```

Internamente `make qemu` ejecuta:

```
1 qemu-system-riscv32 -machine virt -bios none -nographic --no-reboot -smp 2 -kernel
  ↪ kernel
```

El `-smp 2` es lo importante: hace que QEMU emule dos CPUs y, por lo tanto, los dos harts entren a `kernel_main()`. Para salir de QEMU, `Ctrl-A` y después `x`.

1. ¿Por qué aparece la salida mezclada en consola?

Al ejecutar el `kernel.c` sin modificar, la salida que obtuve en uno de mis casos fue:

```
Hello fHello frromom other cpu!
cpu 0!
```

Si uno trata de desentrelazar carácter por carácter, lo que aparece son los dos mensajes esperados (`Hello from cpu 0!` y `Hello from other cpu!`) intercalados. Esto se debe a que las dos CPUs estuvieron escribiendo sobre la misma UART al mismo tiempo y los caracteres terminaron mezclados, que es exactamente el comportamiento que el README anticipa.

Lo que está pasando en el código original es bastante directo en términos de concurrencia: hay dos harts ejecutando el mismo código en paralelo (*interleaving* por paralelismo, no por interrupciones ni por cambios de contexto) y los dos escriben sobre el mismo recurso, la UART. Eso es una *condición de carrera*, y la salida mezclada en pantalla es justo la manifestación visible.

2. Modificación de la función `kernel_main()` para evitar la salida mezclada

El problema no está en `console_putc()` en sí: esa función espera a que el transmisor de la UART esté libre y escribe un carácter, lo cual es correcto para un carácter aislado. Lo que no es atómico es la cadena completa: entre un carácter y el siguiente de un hart, el otro hart puede entrar y meter los suyos.

Por eso protegí toda la impresión, no solo la escritura de un carácter. El cambio que hice en `kernel_main()` fue:

Fragmento relevante de `entrega/kernel.c`

```
1 void kernel_main(void) {
2     // CAMBIO: proteccion de la impresion con el spinlock para evitar que
3     // los caracteres de ambos harts se intercalen en la UART
4     acquire(&lk);
5
6     if (cpuid() == 0) {
7         console_puts("Hello from cpu 0!\n");
8     } else {
9         console_puts("Hello from other cpu!\n");
10    }
```

No modifiqué nada más: ni el arranque, ni la UART, ni las funciones de consola, ni el linker script. Aproveché la variable `lk` que ya estaba declarada y la usé para que funcione como se esperaba.

Con esto, mientras un hart está adentro de la región crítica el otro queda esperando en `acquire()`. Cada mensaje sale completo. El orden entre los dos sigue siendo no determinista (puede salir primero `cpu 0` o el otro hart), pero eso no es lo que la consigna pide controlar.

Salida esperada con el código corregido

Una ejecución correcta debería mostrar las dos líneas completas, sin caracteres intercalados, en alguno de estos dos órdenes posibles:

```
Hello from cpu 0!
Hello from other cpu!
```

```
Hello from other cpu!
Hello from cpu 0!
```

Conclusión

El cambio en sí son dos líneas, pero la parte interesante fue verificar que tenía sentido: la salida mezclada es una condición de carrera entre dos harts sobre la UART, el spinlock me da exclusión mutua, y el módulo spinlock.c ya resuelve internamente la sincronización en SMP (operación atómica más barrera de memoria). Lo que me llevo de este mini-taller es ver, sobre código real, esa cadena de razonamiento en la práctica.